

Web Engineering — Lab 03 Report

CSS Fundamentals — Box Model, Selectors & Specificity

Student Name: Muhammad Tayyab	Course: Web Engineering (5th Semester)
CMS ID: 023-24-0118	Lab: Week 03: CSS Fundamentals

1. Lab Summary

In Week 03, we learned how CSS styles HTML elements. We moved inline styles into an external styles.css file, explored the 4 layers of the Box Model (content, padding, border, margin), tested different CSS selectors in the scavenger hunt, and experimented with CSS specificity and cascade order to see which rules win when they conflict.

2. Activity A: Inline to External Stylesheet

We took inline-start.html which had inline style="..." attributes everywhere, removed the inline styles, and moved them into an external styles.css linked in the <head> with <link rel="stylesheet" href="styles.css">.

Activity A: styles.css

```
/* styles.css */
body {
  background: #f4f1ea;
  font-family: Georgia, serif;
}

h1 {
  color: navy;
  font-size: 2rem;
}

p {
  color: #333;
  font-size: 1.125rem;
  line-height: 1.6;
}
```

3. Activity B: The CSS Box Model

Every element in HTML is a box with 4 layers:

1. **Content:** The actual text or image (width, height).
2. **Padding:** Space inside the box between text and border.
3. **Border:** The visible line around the box.
4. **Margin:** Space outside the box between this box and other elements.

What I tested:

- Changed padding to 40px: the text moved further inside away from the border.
- Changed border to 8px solid navy: the outline became thicker.
- Changed margin to 40px: the space between Box 1 and Box 2 grew larger.
- In DevTools (F12), the box model diagram matched these values exactly.

4. Activity C: Selector Scavenger Hunt (7 Rules)

We applied 7 different selectors in styles.css and checked their results on the page:

#	Selector Type	CSS Rule	What happened on page
1	Tag	h1 { color: navy; font-size: 2rem; }	Only the became navy and bigger.
2	Class	.note { color: teal; font-weight: bold; }	Only the class paragraph became teal and bold.
3	ID	#alert { background: #fff3cd; padding: 8px; }	Only the #alert paragraph got a yellow box.
4	Attribute	input[type="email"] { border: 2px solid teal; }	Only the email input got a teal border; text input stayed normal.

5	Hover	a:hover { color: crimson; text-decoration: underline; }	Links turn crimson when mouse points at them.
6	Odd	li:nth-child(odd) { background: #e3f2fd; }	1st and 3rd list items got light blue background.
7	Even	li:nth-child(even) { background: #fff3e0; }	2nd and 4th list items got light orange background.

5. Activity D: Specificity Duel & Cascade

We tested 4 conflicting rules on `<main id="main"><p class="intro">I am the intro paragraph.</p></main>`:

- `p { color: green; }` — Specificity: (0, 0, 1)
- `.intro { color: blue; }` — Specificity: (0, 1, 0)
- `p.intro { color: purple; }` — Specificity: (0, 1, 1)
- `#main p { color: navy; }` — Specificity: (1, 0, 1)

Observations from DevTools:

1. **All 4 rules active:** The text was **navy** because `#main p` has an ID selector which has the highest specificity (1,0,1). In DevTools Styles panel, the other three rules had line-throughs.
2. **Commented out `#main p`:** The text turned **purple** because `p.intro` (0,1,1) has higher specificity than `.intro` (0,1,0).
3. **Commented out `p.intro`:** The text turned **blue** because `.intro` (0,1,0) beats the element tag `p` (0,0,1).
4. **Source order test:** When we had `p { color: green; }` and `p { color: crimson; }` with equal specificity, the text was **crimson** because the last rule in the file wins.

6. Key Takeaways

- Always use external CSS files instead of inline styles.
- Use classes for styling and keep specificity flat.
- Don't use IDs in CSS because they have high specificity and are hard to override.
- Avoid using `!important`.